Supplementary Information: Situation engineering is a missing layer for reliable artificial intelligence

Geunsik Lim

Supplementary Note 1: Benchmark schema

Each JSONL item contains id, category, query time, question, claims, required variables, gold action and gold answer. Claims contain text and auditable event metadata.

Supplementary Note 2: Complete aggregate results

Method	Accuracy	SCHR	ECE
SCQA	1.000	0.000	0.053
Latest-mention	0.857	0.143	0.183
Recency-RAG	0.714	0.286	0.144
Lexical-RAG	0.700	0.300	0.094

Supplementary Note 3: Category results

category	Latest-mention	Lexical-RAG	Recency-RAG	SCQA
counterfactual	1.000	1.000	1.000	1.000
hidden_premise	0.000	0.000	0.000	1.000
modality	1.000	1.000	1.000	1.000
observer	1.000	0.530	0.000	1.000
scope	1.000	1.000	1.000	1.000
source_conflict	1.000	1.000	1.000	1.000
temporal	1.000	0.372	1.000	1.000

Supplementary Note 4: Ablations

ablation	accuracy	errors
none	1.000	0
time	0.935	273
source	1.000	0
modality	0.857	600
scope	0.857	600
observer	0.857	600
world	1.000	0
missing	0.857	600

Supplementary Note 5: Reproduction

Run `python code/run_experiments.py --n-per-category 600 --data data --out results`. The command regenerates the dataset and all CSV results deterministically.

Supplementary Note 6: Situation-engineering design checklist

Table 1 Minimum reporting checklist for situation-engineered AI systems.

Component	Questions that should be answered
Situation schema	Which actors, events, valid times, scopes, modalities, observers, worlds and uncertainty variables are represented?
Sensing	Which variables are supplied, retrieved or inferred? How is extraction accuracy measured independently of final answers?
Assembly and update	How are event coreference, supersession, conflict, validity intervals and belief revision handled?
Policy	Under which conditions does the system answer, retrieve, clarify, abstain or act?
Calibration	Does state uncertainty propagate into answer confidence and selective risk?
Observability	Can an auditor recover the active state, provenance, alternatives and policy rationale for each output?
Ablation	Does removing a state variable create the predicted category-specific failure rather than only a generic score reduction?
External validity	Are naturally occurring, multilingual, multimodal, adversarial and distribution-shift settings evaluated?
Governance	Are sensitive state variables minimized, access-controlled, correctable and audited for disparate effects?

Supplementary Note 7: Situation-engineering maturity model

Table 2 Proposed SE0–SE4 maturity levels.

Level	Capability	Falsifiable evidence
SE0	Implicit situation inference only	No external situation representation or state-specific evaluation
SE1	Isolated situation tags	Measured extraction of individual variables such as time, location or modality
SE2	Explicit dynamic state	Provenance-linked state, update rules and temporal or scope consistency tests
SE3	Epistemic control	Missing-variable detection, competing hypotheses and answer/retrieve/clarify/abstain policy
SE4	Adaptive audited situation intelligence	Learned multimodal sensing, multi-agent belief tracking, shift monitoring and independent state audits

Supplementary Note 8: Recommended future benchmark modules

A natural-text extension should separately score event extraction, event coreference, temporal ordering, validity intervals, source and modality classification, scope resolution, observer belief, counterfactual frame selection, missing-variable detection, clarification utility, selective risk and claim-level situation consistency. Fixed-state policy tests and fixed-policy sensing tests are both needed to prevent final accuracy from obscuring the source of improvement.

Supplementary Note 9: Frontier-model evaluation status

The repository includes a frozen temporal SituatedQA smoke-test sample and a multi-provider evaluation harness. No frontier-model outputs are reported in this version because the execution environment contained no provider API credentials and no local model checkpoints. The harness aborts rather than creating placeholder predictions. A publishable revision should run the full licensed split on at least three model families, archive raw responses and record exact model snapshots, dates, prompts, token budgets, costs and failed calls. This distinction is essential: an executable protocol is not an experimental result.

Table 3 Recommended typed interfaces between situation engineering and neighbouring components.

Neighbouring discipline	Input to the situation layer	Output or requirement from the situation layer
Prompt engineering	Task, desired output, constraints and user intent	Required situation variables and clarification policy
Context / retrieval engineering	Candidate claims, passages, timestamps and source metadata	Missing-variable queries, applicability filters and freshness requirements
Memory engineering	Events, commitments, observations and provenance across sessions	Validity, supersession, observer/world tags and forgetting constraints
Tool engineering	Tool schema, preconditions, results, side effects and error states	Permitted calls, required confirmation and normalized state transition
Protocol engineering	Capability negotiation, authorization scopes and resource identifiers	Situation schema version, provenance envelope and uncertainty representation
Harness engineering	Workflow trace, checkpoints, permissions and recovery state	State invariants, action gate and answer/retrieve/clarify/abstain decision
Evaluation engineering	Test scenarios, judges, counterfactual perturbations and production traces	State-specific expected behaviour and decomposed failure labels
Observability engineering	Logs, lineage, timing and causal trace	Active-state snapshot, alternatives, unresolved variables and policy rationale
Serving engineering	Model availability, latency, token, memory and cost budgets	Minimum verification budget and graceful degradation policy
Security engineering	Trust zones, identities, privileges and threat signals	State-dependent authorization and response restrictions

Supplementary Note 10: Interfaces among engineering disciplines

Supplementary Note 11: Engineering anti-patterns

The taxonomy enables failure analysis that is more precise than attributing every problem to "the prompt" or "the model". Common anti-patterns include: (i) context dumping, in which more documents are added without resolving which state is active; (ii) memory accumulation, in which obsolete beliefs are retained without validity or supersession metadata; (iii) tool proliferation, in which capabilities are exposed without preconditions, typed errors or state effects; (iv) harness opacity, in which retries and intermediate actions are not connected to an auditable situation state; (v) evaluation collapse, in which only final answer accuracy is measured and sensing, state update and policy failures are indistinguishable; and (vi) serving-induced semantic degradation, in which cost or latency optimizations silently reduce retrieval depth, truncate decisive evidence or skip verification. A situation-engineered system should

make these trade-offs explicit and test whether degraded resource modes preserve calibrated abstention rather than confident error.

Supplementary Note 12: Minimum cross-layer reporting standard

A complete report should identify: the model and prompt; context and retrieval policy; memory write/read/forget rules; tool schemas and error semantics; protocol and authorization assumptions; harness orchestration and recovery; situation schema, update rules and action gate; evaluation data, judges and confidence intervals; observability and provenance records; and serving budgets, latency, cost and failure modes. Reporting only the model name and prompt is insufficient for agentic systems because performance is jointly determined by the model, environment and scaffolding.